

PRACTICAL-07

AIM: Implementation of making a change problem using dynamic programming

PROGRAM:

```
#include <iostream>

#include <vector>

#include <climits>

using namespace std;

int main() { int
    n, amount;

    cout << "Enter number of coin denominations: ";
    cin >> n;

    vector<int> coins(n);

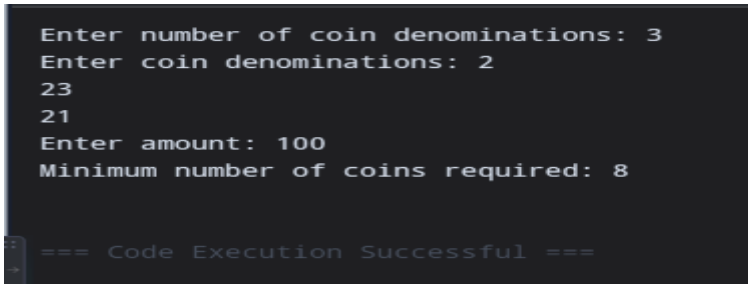
    cout << "Enter coin denominations:
"; for (int i = 0; i < n; i++) { cin >>
coins[i];
}

    cout << "Enter amount: ";
    cin >> amount;

    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
```

```
if (coin <= i && dp[i - coin] != INT_MAX) {  
    dp[i] = min(dp[i], dp[i - coin] + 1);  
}  
}  
}  
  
if (dp[amount] == INT_MAX) cout << "It is not possible  
to make the amount." << endl;  
else  
    cout << "Minimum number of coins required: " << dp[amount] << endl;  
  
return 0;  
}
```

OUTPUT:

```
Enter number of coin denominations: 3  
Enter coin denominations: 2  
23  
21  
Enter amount: 100  
Minimum number of coins required: 8  
  
=== Code Execution Successful ===
```

CONCLUSION:

Dynamic programming solves the making-change problem by storing the minimum coins needed for every amount from 0 to the target. This avoids repeated calculations and gives the minimum number of coins efficient

